International Institute of Information Technology, Hyderabad
Digital VLSI Design

Assignment: RTL to GDSII

4-bit × 4-bit Array Multiplier on SkyWater SKY130

Manas Inamdar
Roll No: 2023102052

Soham Jahagirdar
Roll No: 2023102046

Contents

1 Introduction and Methodology 1
2 Design and Verification (Part A) 1
3 Logic Synthesis (Part B) 3
4 Static Timing Analysis (Part C) 4
5 Physical Design (Part D) 5
6 Answers to the Questions 7
7 Observations and Conclusions 9

1 Introduction and Methodology

For this assignment we designed a registered 4-bit × 4-bit unsigned array multiplier in Verilog and took it through a complete open-source RTL-to-GDSII flow on the SkyWater SKY130 process (`sky130_fd_sc_hd` standard-cell library). The design was verified exhaustively at every stage, synthesized to standard cells, timed with static timing analysis, and placed and routed to a final layout that passes timing, DRC and LVS signoff.

All tools come from the IIC-OSIC-TOOLS Docker image `hpretl/iic-osic-tools:2026.09`: Icarus Verilog 14.0, Yosys 0.69, OpenSTA 3.1.0, LibreLane 3.1.0.dev3 (with OpenROAD, Magic 8.3, KLayout 0.30.12 and Netgen 1.5). The repository README lists the exact command of every step in order.

1.1 Design Flow

The flow consists of the five stages shown in Figure 1, one for each part of the assignment.

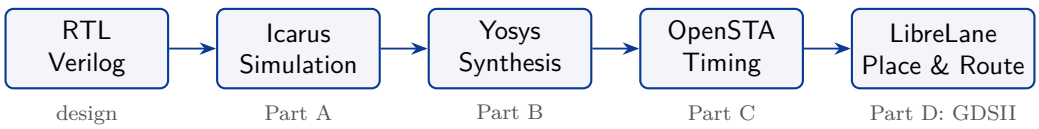


Figure 1: RTL-to-GDSII flow used in this assignment

2 Design and Verification (Part A)

2.1 RTL Design

The module `full_adder` is written with two continuous assignments:

$$sum = a \oplus b \oplus cin \tag{1}$$

$$cout = ab + a\,cin + b\,cin \tag{2}$$

The top module `mult_array` registers the operands `a` and `b` into `a_reg` and `b_reg` on every rising clock edge. From the registered values it forms the 16 partial products

$$pp_i[j] = \texttt{a_reg}[j] \cdot \texttt{b_reg}[i], \qquad i, j \in \{0, 1, 2, 3\} \tag{3}$$

with explicit AND assignments, and adds them with 12 `full_adder` instances arranged as three 4-bit ripple-carry rows, wired exactly as in the connection table of the assignment (Figure 2). The 8-bit result is registered into `p`. All 16 flip-flops are cleared by the asynchronous active-low reset `rst_n`, and the design does not use the `*` operator. Operands captured at edge 1 produce the product at edge 2, which gives a latency of two clock cycles and a throughput of one result per cycle.

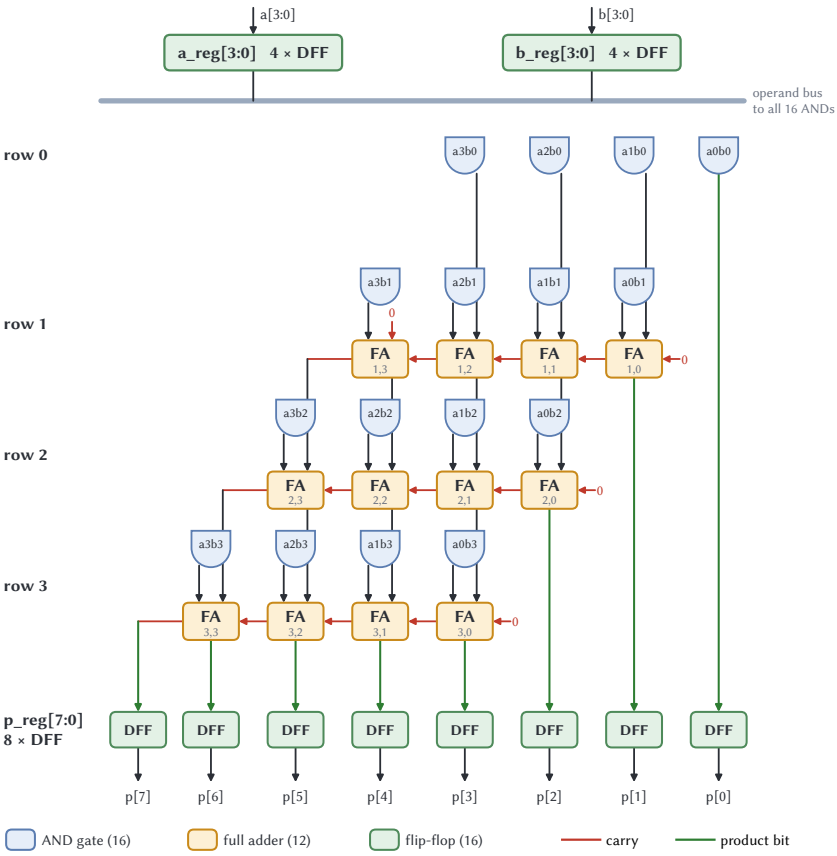


Figure 2: Block diagram of `mult_array`: input registers, 16 AND gates, 12 full adders FA(i,j) in rows 1–3, and the 8-bit output register. Columns correspond to bit weights, so each sum passes straight down to the next row while carries ripple from right to left

2.2 Testbench and Results

The testbench `tb_mult` generates a 10 ns clock and holds `rst_n` low for the first two rising edges. It then applies all 256 combinations of `a` and `b` on falling clock edges, waits for two rising edges and compares `p` with `a*b` (the only place the multiplication operator is used). It prints every mismatch, exactly one summary line, and writes the waveform to `mult.vcd`. For the error test (task 6) we changed the sum equation to `a ^ b` in a copy of `full_adder.v`:

```
$ iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v && vvp sim
PASS: 256 vectors checked, 0 errors

# error test: sum = a ^ b
MISMATCH: a=3 b=3 expected=9 actual=5            (95 mismatch lines in total)
FAIL: 256 vectors checked, 95 errors
```

Dropping `cin` from the sum corrupts every product that needs a carry into a sum bit, and 95 of the 256 products fail. With the original equation restored the design passes again.

2.3 Hand-worked Example

We used the roll number of Manas Inamdar, 2023102052. Its last digit is 2, so $A = 2 + 5 = 7$, and because $A = 7$ the assignment sets $B = 9$. The left box follows the layout of Section 2 of the assignment; the right box traces the same product through the rows of the array.

```
          0 1 1 1   A = 7
      x   1 0 0 1   B = 9
  ---------------
          0 1 1 1   pp0  (b0 = 1)
        0 0 0 0     pp1  (b1 = 0)
      0 0 0 0       pp2  (b2 = 0)
    0 1 1 1         pp3  (b3 = 1)
  ---------------
  0 0 1 1 1 1 1 1   P = 63
```

```
row 0  pp0 = 0111            p[0]=1  r0 = 0011
row 1  0000 + 0011 = 0011    p[1]=1  r1 = 0001
row 2  0000 + 0001 = 0001    p[2]=1  r2 = 0000
row 3  0111 + 0000 = 0111    p[3]=1
       p[7:4] = {c3[4], s3[3:1]} = 0011

P = 0011 1111 = 63 = 7 x 9
```

2.4 Waveform

Figure 3 shows the same transaction in the RTL simulation. The operands appear on a falling edge, are captured on the next rising edge, and the product is registered one cycle later.

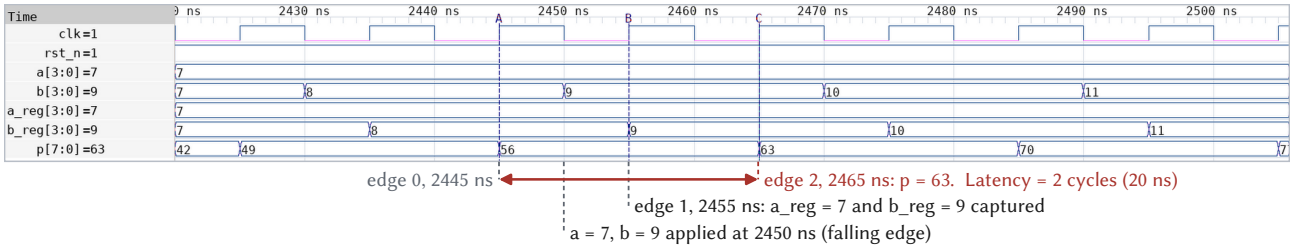


Figure 3: GTKWave view of `mult.vcd` for $A = 7$, $B = 9$ (colours inverted for print). The operands are applied at 2450 ns, captured at edge 1 (2455 ns), and $p = 63$ is registered at edge 2 (2465 ns): a latency of two clock cycles. Markers A, B and C sit on edges 0, 1 and 2

3 Logic Synthesis (Part B)

The script `synth/synth.tcl` performs the seven required Yosys steps with the `tt_025C_1v80` Liberty library: reading the RTL, generic synthesis with flattening (`synth -flatten`), flip-flop mapping (`dfflibmap`), gate mapping (`abc`), removal of unused wires (`opt_clean -purge`), the cell and area report (`stat -liberty`), and the gate-level netlist (`write_verilog`). The mapped netlist was then simulated with the same testbench and the SKY130 functional cell models (`-DFUNCTIONAL`, unit delay); it passes all 256 vectors.

Metric	Value
Total number of cells	63
Number of flip-flops	16 (`dfrtp_1`)
Total cell area	$740.7104\,\mu m^2$
Flip-flop area	$16 \times 25.024 = 400.384\,\mu m^2$
Flip-flop area (% of total)	54.05 %
Three most-used cell types	`dfrtp_1` (16), `nand2_1` (8), `a21oi_1` (6)
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

Table 1: Synthesis results from `synth/mult_array_stat.rpt`

Cell	No.	Cell	No.	Cell	No.	Cell	No.
`dfrtp_1`	16	`and2_0`	2	`and3_1`	1	`nor2_1`	1
`nand2_1`	8	`clkinv_1`	2	`and4_1`	1	`nor2b_1`	1
`a21oi_1`	6	`lpflow_isobufsrc_1`	2	`nand2b_1`	1	`nor3_1`	1
`xnor2_1`	4	`xnor3_1`	2	`nand4_1`	1	`o2111ai_1`	1
`maj3_1`	3	`a22oi_1`	1	`o21ai_0`	1	`o311ai_0`	1
`nand3_1`	3	`xor3_1`	1				
`xor2_1`	3					**Total**	**63**

Table 2: All 63 synthesized cells (names without the `sky130_fd_sc_hd__` prefix)

4 Static Timing Analysis (Part C)

The script `sta/sta.tcl` reads the Liberty library and the netlist, links `mult_array`, defines `clk` with period T, applies 1.0 ns input and output delays, prints the worst setup slack and writes the critical-path report. At $T = 10$ ns the worst slack (WNS) is 7.41 ns, so

$$T_{\min} = T - \mathrm{WNS} = 10 - 7.41 = 2.59 \text{ ns}, \tag{4}$$

$$F_{\max} = \frac{1000}{T_{\min}} = \frac{1000}{2.59} = 386.10 \text{ MHz}. \tag{5}$$

Re-running at $T = 2.59$ ns reports a worst slack of -0.00 ns (0.000 ns in the detailed report): rounding $T_{\min}$ to 0.01 ns lands exactly on the limit, and at 2.60 ns the slack is $+0.01$ ns.

Metric	Value
Worst slack at $T = 10$ ns	7.410 ns
$T_{\min}$	2.59 ns
$F_{\max}$	386.10 MHz
Slack at $T = T_{\min}$	0.000 ns (≈ 0)
Critical path start point	`_087_`, input register `a_reg[1]` (`dfrtp_1`)
Critical path end point	`_100_`, output register `p[6]` (`dfrtp_1`)
Number of cells on the critical path	10 (8 logic cells, launch and capture flip-flops)
Latency $= 2 \times T_{\min}$	5.18 ns
Throughput	386.10 million results per second

Table 3: Static timing results at the typical corner (ideal clock, no wire parasitics)

Instance	Cell	Arrival (ns)	Function in the RTL
_087_	dfrtp_1	0.398	launch flip-flop, a_reg[1]
_039_	clkinv_1	0.471	$\overline{\texttt{a_reg[1]}}$
_047_	o311ai_0	0.827	partial products merged with FA(1,0), FA(1,1)
_050_	a21oi_1	0.999	carry of FA(2,0), c2[1]
_063_	maj3_1	1.381	carry of FA(2,1), c2[2]
_064_	xnor2_1	1.541	sum of FA(2,2), $\overline{\texttt{s2[2]}}$
_073_	maj3_1	1.924	carry of FA(3,1), $\overline{\texttt{c3[2]}}$
_075_	maj3_1	2.281	carry of FA(3,2), $\overline{\texttt{c3[3]}}$
_079_	xnor3_1	2.457	sum of FA(3,3), s3[3] = p[6]
_100_	dfrtp_1	setup 0.134	capture flip-flop, p[6]

Table 4: Cells on the critical path at $T = 2.59$ ns. The last column was found by simulating the netlist and the RTL for all 256 register values and matching the truth table of each net to the RTL signals

In RTL terms the path runs a_reg[1] → FA(1,0) → FA(1,1) → FA(2,0) → FA(2,1) → FA(2,2) → FA(3,1) → FA(3,2) → FA(3,3) = p[6]. It passes through **eight full-adder stages, exactly the $2N = 8$ estimate**: first down the rows, then along the carry chain of row 3. Six of the eight logic cells each compute one full-adder output. FA(1,0) has a constant-zero carry input, so synthesis merged it with the partial-product gates and FA(1,1) into the inverter and the o311ai cell.

5 Physical Design (Part D)

The configuration pnr/config.json sets DESIGN_NAME, VERILOG_FILES (the RTL), CLOCK_PORT, FP_SIZING = "absolute", DIE_AREA = [0, 0, 150, 150], the PDK and cell library, and CLOCK_PERIOD. The first period was

$$1.2 \times T_{\min} = 1.2 \times 2.59 = 3.108 \text{ ns} \rightarrow 3.5 \text{ ns (next 0.5 ns)}. \tag{6}$$

LibreLane signs off setup and hold timing at nine corners (typical, slow and fast process, each with nominal, minimum and maximum parasitics). Every failing run failed only on setup at the slow ss_100C_1v60 corner, with DRC and LVS clean, so the period was raised by 0.5 ns per run until timing closed (Table 5).

5.1 LibreLane Runs

Run	Period (ns)	Worst slack (ns)	Typical slack (ns)	DRC	LVS	Result
1	3.5	−2.029	+0.587	0	0	setup violation
2	4.0	−1.529	+1.087	0	0	setup violation
3	4.5	−1.028	+1.588	0	0	setup violation
4	5.0	−0.528	+2.088	0	0	setup violation
5	5.5	−0.146	+2.542	0	0	setup violation
6	6.0	−0.151	+2.887	0	0	setup violation
7	**6.5**	**+0.041**	+3.260	0	0	**passed**

Table 5: All LibreLane runs. The worst-corner slack is always set by ss_100C_1v60

5.2 Final Results

The post-route maximum frequency follows from the final period and setup slack:

$$F_{\text{max,post-route}} = \frac{1000}{\text{CLOCK_PERIOD} - \text{slack}} = \frac{1000}{6.5 - 0.041} = 154.83 \text{ MHz}. \tag{7}$$

Metric	Value
`CLOCK_PERIOD` used	6.5 ns
Number of runs needed	7 (3.5 to 6.5 ns in 0.5 ns steps)
Core area	17,759.5 µm^2 (die 22,500 µm^2)
Cells after synthesis (Part B)	63
Cells after place and route	1,893
Kinds of cells added	tap, decap, fill, timing-repair and clock-tree buffers
Wirelength	2,334 µm
Setup slack (hold slack)	+0.041 ns (+0.129 ns)
Post-route F_{max}	154.83 MHz
F_{max} from Part C	386.10 MHz
DRC errors	0 (Magic 0, KLayout 0, detailed router 0)
LVS errors	0 (Netgen: circuits match uniquely)

Table 6: Physical design results of the final run (`pnr/metrics.json`)

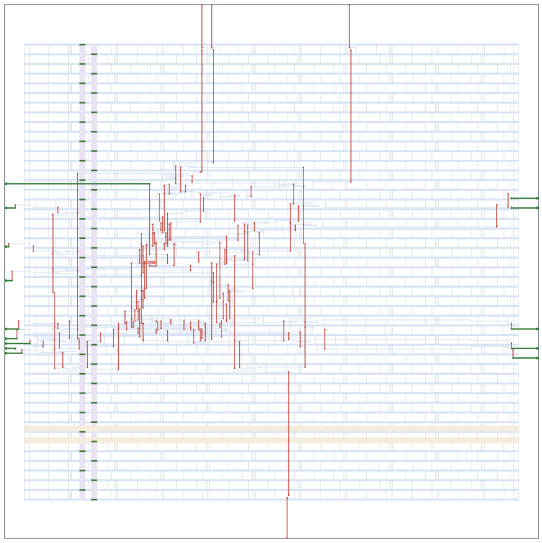
Kind of cell	Library cells	Count
Logic and flip-flops	from LibreLane's own synthesis	68
Decoupling capacitors	`decap_3`, `decap_4`, `decap_6`, `decap_8`, `decap_12`	1,301
Fill	`fill_1`, `fill_2`	257
Well taps	`tapvpwrvgnd_1`	245
Timing-repair buffers	`buf_2` (17), `clkdlybuf4s50_1` (2)	19
Clock-tree buffers	`clkbuf_16`	3
Total		**1,893**

Table 7: Cells in the final layout, counted from the routed netlist. LibreLane runs its own synthesis, which gives 68 logic cells and flip-flops instead of the 63 of Part B

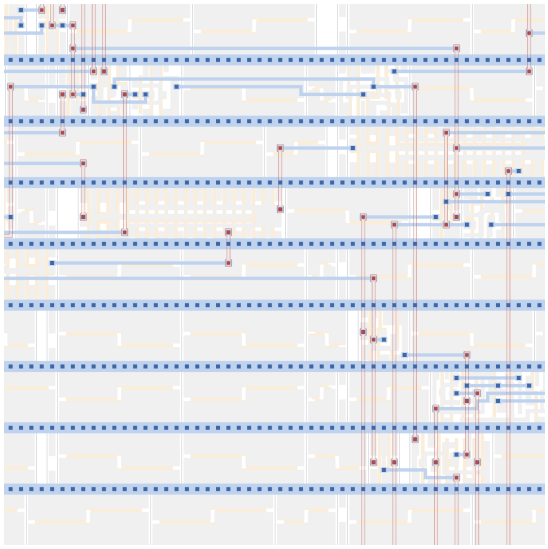
From these results we observed that:

- The post-route F_{max} is 2.5 times lower than in Part C. At the same typical corner as Part C the routed design would allow $1000/(6.5 - 3.260) = 308.6$ MHz; the rest of the reduction comes from slow-corner signoff (Section 6.4).
- Of the 1,830 additional instances, 1,803 are physical-only cells without a logic function: the rows are only 7 % utilised and every empty site is filled. The flow also added 19 repair buffers and 3 clock buffers.
- The final metrics report 11 maximum-slew violations, all at the slow corner and all on the reset net: one buffer drives ten `RESET_B` pins with a 0.778 ns transition against a 0.75 ns limit (28 ps over). Reset recovery and removal still pass at that corner (+4.01 ns and +0.97 ns), and slew is not one of the assignment's acceptance criteria (0 DRC errors, 0 LVS errors, setup slack ≥ 0).

5.3 Layout



(a) Whole 150 µm × 150 µm die: cell outlines, met1 rails, met4/met5 power straps, met2 (red) and met3 (green) routing, and I/O pins on the edges



(b) 24 µm × 24 µm detail at the centre of the logic: cell rows between met1 power rails, local interconnect, met1 wires and met2 risers with vias

Figure 4: Final layout `pnr/mult_array.gds` in KLayout 0.30.12 (subset of the SKY130 layers)

6 Answers to the Questions

6.1 Logic Cells after Synthesis

Synthesis produced 63 cells, of which 16 are flip-flops, so **47 cells** implement the logic rather than $16 + 12 = 28$. The RTL gates are not mapped one to one: after flattening, ABC optimises the whole logic network and selects whichever SKY130 cells are cheapest. A full adder has two outputs, and ABC does not use the two-output SKY130 `fa` cell, so each adder needs at least a carry cell (`maj3`, or an `a21oi`/`nand2` combination) and a sum cell (`xor3`/`xnor3`, or two two-input XOR/XNOR cells). Conversely, adders with constant inputs ($c_i[0] = 0$ and $r_0[3] = 0$) reduce to half adders or single gates, and many AND gates are absorbed into complex cells such as `a21oi`, `o311ai`, `o2111ai`, `nand3` and `nand4`. Only three `maj3` and three three-input XOR/XNOR cells remain as complete full-adder functions.

6.2 Critical Path

The critical path starts at the input register `a_reg[1]` (flip-flop `_087_`) and ends at the output register bit `p[6]` (`_100_`). It contains 10 cells, the two flip-flops and eight logic cells, with a data arrival time of 2.457 ns (Table 4). Traced back to the RTL it crosses FA(1,0), FA(1,1), FA(2,0), FA(2,1), FA(2,2), FA(3,1), FA(3,2) and FA(3,3): **eight full-adder stages, matching the** $2N = 8$ **estimate exactly**. The cell count equals the stage count because most stages map to a single complex cell, while the first two stages are merged with the AND gates into three cells.

6.3 Flip-flop Share of the Area

The 16 flip-flops occupy 400.384 of 740.710 µm^2, or **54.05 %** of the cell area. A `dfrtp_1`, a D flip-flop with asynchronous reset, is a large cell of 25.024 µm^2: 3.5 times the average logic cell ($340.33/47 = 7.24$ µm^2). The number of registers grows linearly with the operand width ($4N = 16$ bits for a, b and the $2N$-bit product), whereas the array logic grows as N^2. At $N = 4$ the array is so small that the fixed register cost dominates; in a wider multiplier the logic would take over.

6.4 Post-route Maximum Frequency and Added Cells

Part C assumes ideal conditions: the typical corner, no wire parasitics, an ideal clock and the bare netlist. LibreLane signs off the real layout instead:

- at the slow corner `ss_100C_1v60` (1.60 V, 100 °C), where cells are about 2.4 times slower: the launch flip-flop's clock-to-output delay is 0.97 ns instead of 0.40 ns, and each `maj3` takes about 0.92 ns instead of 0.38 ns;
- with extracted wire resistance and capacitance;
- with a real clock tree, which adds 0.55 ns of insertion delay, and a 0.25 ns clock uncertainty;
- with a longer setup time (0.29 ns).

Together these give **154.83 MHz instead of 386.10 MHz**; measured at the typical corner, the routed design would reach 308.6 MHz. The flow added four kinds of cells:

- **Well taps (245)** tie the n-well and substrate to VPWR and VGND at regular intervals to prevent latch-up.
- **Decoupling capacitors (1,301)** sit between VPWR and VGND, supply local current spikes and reduce supply noise.
- **Fill cells (257)** close the remaining gaps so that wells, implants and power rails are continuous and density rules are met.
- **Buffers (22):** 19 repair buffers fix slew, capacitance and fanout on long or heavily loaded nets such as the reset and input nets, and three `clkbuf_16` cells form the clock tree that delivers a sharp clock edge to all 16 flip-flops with little skew.

6.5 DRC and LVS

DRC (Magic and KLayout) checks that the layout geometry obeys the foundry rules on every layer: minimum width and spacing, via and contact enclosure, minimum area, density and antenna rules. LVS (Netgen) extracts the devices and connections from the layout and checks that they form exactly the circuit of the netlist, with the same cells, nets and pins and no shorts or opens. Both must pass before manufacture. A DRC error means the masks cannot be printed reliably, which causes shorts, opens or low yield, and the foundry will reject the design. An LVS mismatch means the fabricated chip would be a different circuit from the one that was simulated and timed, which invalidates all of the verification. Our final layout has no DRC and no LVS errors.

6.6 Scaling to Eight Bits

Using the formulas of Section 2 of the assignment with the delay per stage measured here ($2.457 - 0.398 = 2.059$ ns for eight stages, about 0.257 ns per stage):

$$T_{\min}(N) \approx t_{\text{clk-q}} + 2N \cdot t_{\text{stage}} + t_{\text{setup}} = 0.398 + 2N \times 0.257 + 0.134 \text{ ns}. \tag{8}$$

Quantity	$N = 4$	$N = 8$	Ratio
AND gates, N^2	16	64	4.0
Full adders, $N(N-1)$	12	56	4.7
Flip-flops, $4N$	16	32	2.0
Critical path, $\approx 2N$ full-adder delays	8	16	2.0
$T_{\min}$ (ns)	2.59	4.65	1.8
$F_{\max}$ at the typical corner (MHz)	386	215	0.56

Table 8: Estimated effect of doubling the operand width

The number of full adders grows quadratically, but the critical path only doubles because it grows linearly with N. The latency remains two cycles, but each cycle becomes almost twice as long.

7 Observations and Conclusions

7.1 What We Observed

1. **Correct at every level:** the RTL, the synthesized netlist and the routed design all pass 256 of 256 vectors, and the deliberate full-adder fault is detected (95 errors).
2. **Synthesis is not one-to-one:** the 16 AND gates and 12 full adders become 47 logic cells and 16 flip-flops (740.7 µm^2), and the flip-flops take 54 % of the area.
3. **The $2N$ estimate holds:** the pre-layout critical path crosses exactly eight full-adder stages and allows 386.10 MHz at the typical corner.
4. **Signoff corner dominates:** the design closes at 6.5 ns after seven runs (154.83 MHz). Most of the frequency gap comes from slow-corner signoff; wires, the clock tree and repair buffers add the rest.
5. **Reproducible:** re-running the flow from `pnr/config.json` reproduces every committed result exactly.

7.2 Conclusions

Summary

We designed, verified and implemented a registered 4-bit array multiplier from RTL to a DRC- and LVS-clean GDSII layout on SKY130. The final design runs at a 6.5 ns clock (154.83 MHz) with +0.041 ns setup slack, in a 150 µm × 150 µm die with 1,893 cells and 2,334 µm of wire.

7.3 Team Contributions

Team Member Contributions

Manas Inamdar (2023102052):

- Set up the repository and the IIC-OSIC-TOOLS environment
- Wrote the RTL (`full_adder.v`, `mult_array.v`) and the exhaustive testbench (`tb_mult.v`)
- Wrote `synth.tcl` and ran synthesis and gate-level simulation (Part B)
- Wrote `sta.tcl` and ran the static timing analysis (Part C)
- Wrote `config.json` and carried out the seven LibreLane runs (Part D)

Soham Jahagirdar (2023102046):

- Reproduced the complete flow and verified every committed result
- Strengthened the testbench and simulated the routed netlist
- Traced the critical-path cells back to the full adders of the array
- Explored physical-design timing optimization
- Prepared the README, the figures and this report

Joint contributions:

- Analysis of the results and answers to the questions
- Final review of the code, results and report

Declaration on the use of AI tools

We used an AI assistant (Claude, Anthropic) to cross-check our flow. It re-ran the simulations, synthesis, static timing analysis and LibreLane runs, mapped the critical-path cells to the RTL, produced the waveform figure, the block diagram and the layout renders, and helped draft the README and this report. Every number in this report comes from the open-source tools listed in Section 1. Both team members have reviewed every file, result and answers.